

TIDE: Trend-Informed Delayed-label Eviction

Benchmark results

Cache replacement project

September 27, 2026

Summary

TIDE is an ARC cache in which a small neural network, trained online, chooses which item to evict. ARC’s own rule picks the list to evict from. A $12 \rightarrow 64 \rightarrow 1$ MLP, written in pure NumPy, then scores that list’s 4 least-recently-used keys, its most-recently-used key, and the incoming key. Choosing the incoming key means it is not cached at all (bypass). The network predicts $\log(1 + d/c)$, where d is the time until the key is requested again and c is the cache size. Every scored key gets its true d as a training label when it is next requested, or a censored label after $W = 8c$ requests, so the model keeps learning as the cache runs. A shadow copy of plain ARC runs alongside; whenever TIDE falls behind it, TIDE hands decisions back to ARC.

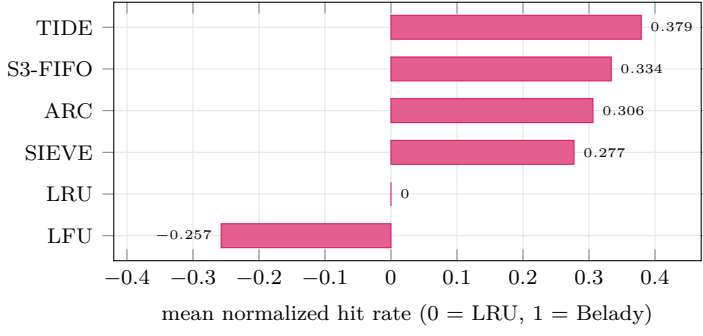
- **Best overall hit rate.** Mean normalized hit rate is 0.379 for TIDE, against 0.334 for S3-FIFO, 0.306 for ARC and -0.257 for LFU. Normalized hit rate is $(r - r_{\text{LRU}})/(r_{\text{Belady}} - r_{\text{LRU}})$ averaged over 42 cells: 0 means no better than LRU, 1 means as good as the offline optimum. TIDE has the highest hit rate in 24 of 43 cells.
- **Close to ARC’s floor.** $\text{TIDE} \geq \text{ARC}$ in 34 of 43 cells. Its worst cell is -0.54 points (phases, $c = 20181$).
- **Trend workloads, best gain over ARC:** cycle $+34.5$, phases $+10.6$, lifecycle $+0.9$ points. The adversarial loop gains $+48.6$ points. TIDE realizes 37% of the gap between ARC and the TIDE oracle on the trend traces.
- **Fast.** A decision takes a median of $15\mu\text{s}$ at p50 and at most $30\mu\text{s}$ at p99. Throughput is 45k–123k requests/s, and hits never call the model.

Setup

Policies. LRU; LFU (standard in-cache LFU, which forgets counts on eviction); ARC; S3-FIFO; SIEVE; TIDE (mean over 3 seeds); the *TIDE oracle*, which has the same design but scores keys with the true next-request time, giving the design’s ceiling; and Belady’s MIN, the offline optimum. With bypass allowed, Belady is not a strict upper bound.

Traces. 15 traces in total. There are 12 existing CSV traces of 100k requests: Zipf, web traffic with a drifting hot set, periodic, bursty, adversarial, Gaussian, uniform, and Zipf with $\alpha \in \{0.5, \dots, 1.5\}$. There are also 3 generated 1M-request trend traces: *lifecycle*, where items are born and their popularity rises then decays; *cycle*, where 40 groups of 100 keys recur in a fixed order over a Zipf background; and *phases*, which runs Zipf A, a scan, a loop and Zipf B, twice. Cache sizes are 0.1%, 1% and 10% of each trace’s distinct keys (minimum 16).

Results



Policy	Norm.	Wins	Mean hit%
TIDE	0.379	24	46.09
S3-FIFO	0.334	11	44.00
ARC	0.306	3	43.36
SIEVE	0.277	2	42.92
LRU	0.000	1	38.85
LFU	-0.257	2	35.24

Figure 1: Overall ranking across all cells. “Wins” counts the cells where the policy has the highest hit rate among the six contenders.

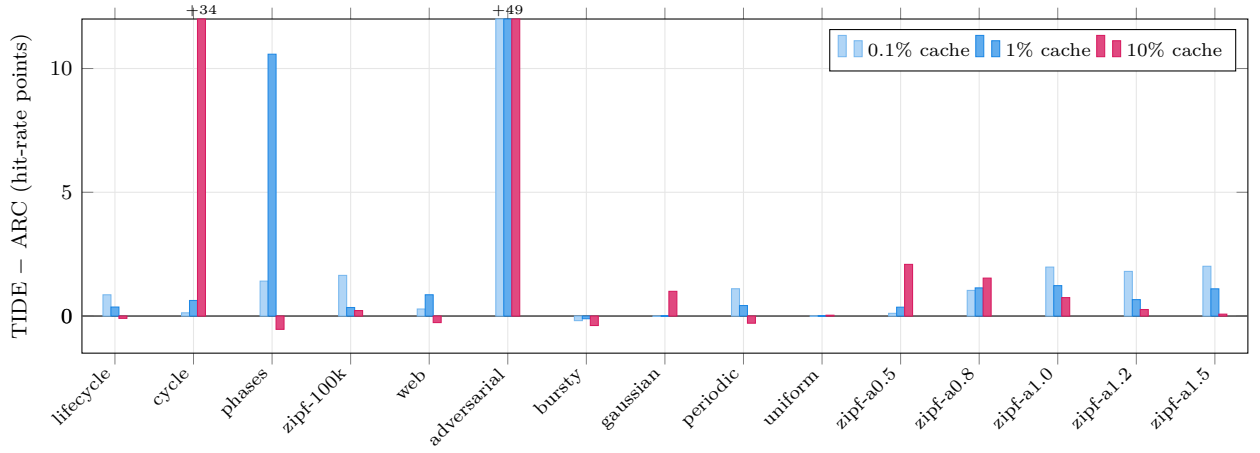


Figure 2: Gain of TIDE over ARC per trace and cache size. Bars above 12 points are clipped and labelled with their value.

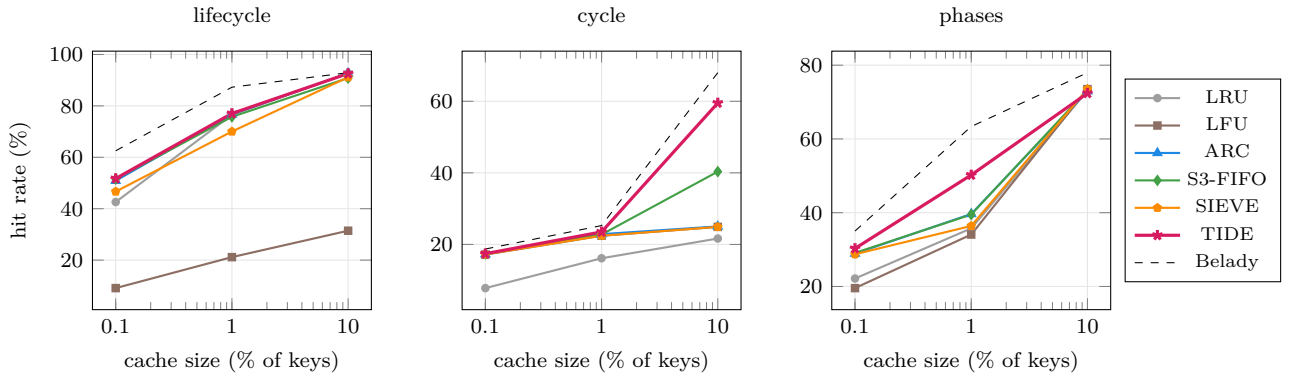


Figure 3: Hit rate vs. cache size on the three 1M-request trend traces.

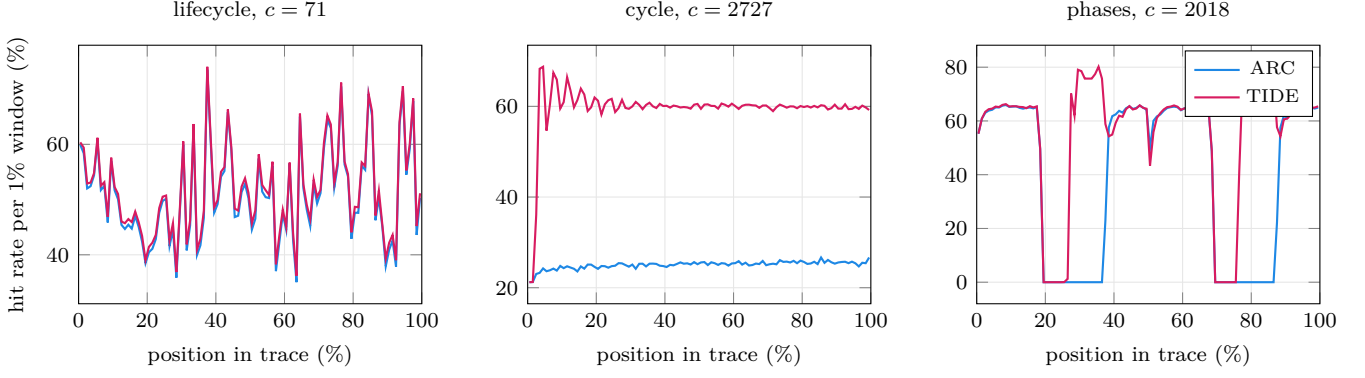


Figure 4: Learning live: hit rate over time for ARC and TIDE, at the cache size where TIDE gains most on each trend trace. TIDE starts out as plain ARC and switches to its model once it has ≥ 2048 labels and $t \geq W$.

Trace	Requests	c	ARC req/s	TIDE req/s	p50 (μ s)	p99 (μ s)	Training %
zipf-100k	0.1M	185	3,995k	110k	14.6	22.9	21
web	0.1M	50	1,864k	45k	14.8	20.2	19
lifecycle	1M	708	3,709k	123k	15.2	30.0	20
cycle	1M	273	2,208k	76k	13.9	21.9	27
phases	1M	2018	2,516k	84k	14.3	21.8	19

Table 1: Speed, measured serially at the 1% cache size in pure Python/NumPy. Latency is per eviction decision and covers features, label snapshot and inference; training steps are excluded. Training % is the share of total runtime spent in online training.

Caveats

- LFU here is the standard in-cache LFU. An LFU with global, never-forgotten counts (unbounded meta-data) beats TIDE on the i.i.d. Zipf traces; it is in `bench_results.csv` as `LFU*` and is left out of the contenders.
- The trend generators and their parameters were written for this project and fixed before tuning; tuning used the same settings on every trace. TIDE’s settings ($\delta = 0.1$ for tail keys, 0.2 for MRU/bypass, one training step per 32 labels, 64 hidden units) come from a single sweep over all traces.
- Pure Python runs 20–70 $\times$ below plain ARC’s raw throughput; a Numba or C hot path would close that.
- TIDE missed some internal targets: worst cell -0.54 (target ≥ -0.5), lifecycle gain < 1 point, and training at about 21% of runtime (target $\leq 15\%$).

Trace	Size%	c	LRU	LFU	ARC	S3-FIFO	SIEVE	TIDE	Oracle	Belady	Δ ARC
lifecyle	0.1	71	42.60	9.11	50.87	51.53	46.67	51.73	56.26	62.53	+0.86
	1	708	76.56	21.16	76.68	75.75	70.03	77.04	79.86	87.27	+0.36
	10	7084	92.70	31.44	92.68	90.83	91.14	92.58	92.70	92.92	-0.09
cycle	0.1	27	7.79	17.16	17.28	17.38	17.23	17.41	17.95	18.71	+0.13
	1	273	16.13	22.43	22.87	22.76	22.43	23.50	23.61	25.27	+0.63
	10	2727	21.65	24.90	25.04	40.35	24.90	59.52	59.54	67.99	+34.49
phases	0.1	202	22.16	19.55	28.90	29.10	28.74	30.31	31.65	35.03	+1.41
	1	2018	35.76	34.08	39.65	39.50	36.42	50.23	56.89	63.37	+10.58
	10	20181	73.09	73.50	72.93	73.27	73.50	72.39	74.15	77.96	-0.54
zipf-100k	0.1	18	35.46	47.55	47.57	49.23	49.04	49.21	51.18	53.50	+1.64
	1	185	59.75	67.03	67.63	67.73	67.02	67.97	68.59	71.70	+0.34
	10	1849	74.75	77.37	77.81	77.40	77.37	78.03	78.92	80.93	+0.22
web	0.1	16	3.33	1.33	4.53	5.02	4.63	4.81	18.66	21.48	+0.28
	1	50	10.30	3.36	14.08	14.56	13.67	14.94	31.30	38.86	+0.86
	10	499	67.64	30.54	71.39	70.72	67.20	71.13	72.97	78.89	-0.26
adversarial	0.1	16	0.00	0.00	0.00	0.00	0.00	48.55	51.60	49.99	+48.55
bursty	0.1	16	84.56	84.56	84.56	84.57	84.56	84.38	85.00	85.27	-0.19
	1	79	84.66	84.67	84.67	84.66	84.67	84.56	85.41	86.27	-0.11
	10	789	85.70	85.72	85.72	85.71	85.71	85.34	87.25	89.44	-0.38
gaussian	0.1	16	0.28	0.46	0.47	0.54	0.46	0.46	5.15	6.28	-0.02
	1	88	1.47	1.96	1.98	2.00	1.99	1.98	10.83	15.97	+0.00
	10	876	14.81	16.85	16.95	17.18	17.11	17.94	32.34	47.45	+1.00
periodic	0.1	16	10.02	1.21	11.48	12.29	11.35	12.59	32.56	36.44	+1.10
	1	89	50.62	4.14	66.43	63.69	60.62	66.85	71.69	76.02	+0.42
	10	893	79.71	31.45	79.74	79.23	78.41	79.45	81.67	85.38	-0.29
uniform	0.1	16	0.19	0.17	0.18	0.19	0.17	0.17	3.80	4.79	-0.01
	1	100	1.01	0.98	1.04	1.05	0.98	1.04	8.53	13.14	+0.00
	10	1000	10.02	9.97	10.01	9.97	9.97	10.04	25.35	40.00	+0.03
zipf-a0.5	0.1	16	0.40	2.01	2.25	2.66	2.06	2.36	6.24	7.19	+0.11
	1	100	2.36	6.03	6.56	7.13	6.24	6.92	12.98	18.21	+0.36
	10	998	17.84	22.97	23.39	24.58	23.00	25.48	32.32	48.11	+2.09
zipf-a0.8	0.1	16	4.33	12.55	13.34	14.62	14.10	14.38	17.93	19.92	+1.03
	1	97	15.44	26.47	26.37	27.33	26.98	27.50	30.43	36.75	+1.14
	10	968	43.09	50.50	50.33	51.39	50.59	51.86	54.20	65.71	+1.53
zipf-a1.0	0.1	16	18.18	31.46	31.02	33.01	32.59	33.00	35.97	38.50	+1.98
	1	86	37.34	48.71	48.27	49.19	49.69	49.49	51.13	56.62	+1.23
	10	857	65.40	71.31	70.91	71.38	71.31	71.65	72.38	79.47	+0.74
zipf-a1.2	0.1	16	33.34	44.92	45.78	47.53	46.95	47.58	49.81	51.85	+1.80
	1	88	52.72	61.43	61.61	62.10	61.95	62.27	63.12	66.83	+0.66
	10	884	71.87	75.16	75.37	75.54	75.16	75.63	76.76	82.09	+0.26
zipf-a1.5	0.1	16	72.82	79.51	78.11	79.94	79.77	80.12	81.43	82.94	+2.01
	1	28	79.27	84.80	83.53	84.49	84.60	84.63	85.62	87.30	+1.10
	10	280	93.38	94.75	94.69	94.70	94.75	94.76	94.96	95.94	+0.08

Table 2: Hit rate (%) for every cell. Bold marks the best of the six contenders. Oracle is the TIDE oracle (true next-request scores); Δ ARC is TIDE - ARC. TIDE is the mean over 3 seeds.